MicroProof

Zero-Attention Local Micro-Contract & Proof Ledger — System Architecture & Engineering Spec

Document Type: Technical Architecture Specification

Target Platform: Android (React Native / Expo)

Security: Ed25519 P2P Handshake

1. EXECUTIVE SUMMARY & PROBLEM SPACE

MicroProof is a lightweight, offline-first mobile application designed to eliminate low-stakes civil friction between individuals in physical proximity. By replacing informal verbal agreements with cryptographically signed, timestamped micro-contracts and bills of sale, MicroProof provides legally robust proof of agreements—such as tool loans, private vehicle sales, sub-lease inspections, and cash handoffs—without reliance on servers, cloud accounts, or payment processing middlemen.

2. HIGH-LEVEL CRYPTOGRAPHIC HANDSHAKE ARCHITECTURE



3. KEY SYSTEM PILLAR ANALYSIS

Zero Cloud & Local Security

Private keys are generated upon initial boot inside the hardware-backed Android Keystore using Expo SecureStore. Private key material never exits the enclave; only public keys are embedded into agreement payloads.

Air-Gapped Optical Signing

By compressing payloads into dynamic QR codes, contracts are transferred and mutual-signed between devices without cellular signal, Wi-Fi, or Bluetooth pairing overhead.

Legal Evidentiary Export

Generates self-contained PDF certificates containing full transaction metadata, photo hash verification, GPS coordinate signatures, and public keys formatted for small claims admissibility.

Zero Payment Liability

Acts strictly as an immutable receipt referee for physical transactions settled via cash or bank apps, avoiding escrow, KYC, and payment processing regulations.

4. CONTRACT TEMPLATE CATALOG & SCHEMA MAPPING

MicroProof supports standardized, pre-built schemas configured to capture critical legal parameters while keeping user entry time under 45 seconds.

Template Type	Key Input Fields	Default Clauses Included	Verification Protocol
PRIVATE BUY / SELL	Item Name, Serial/IMEI, Price, Cash/Bank tag	Sold "As-Is", Ownership transfer, Payment received	Dual QR + Photo Handoff
EQUIPMENT LOAN	Item Description, Return Date, Deposit Value	Return condition agreement, Damage penalty clause	Dual QR + Photo Proof
PROPERTY CHECK	Location/Unit, Pre/Post condition notes	Existing defects acknowledgment, Deposit clearance	Multi-Photo Hash Array
EXPENSE SPLIT	Amount, Repayment Due Date, Reason	Debt acknowledgment, Non-interest clause	Dual QR Handshake

5. SQLITE LOCAL DATABASE SCHEMA DEFINITION

Column Name	Data Type	Constraints	Description
`id`	TEXT (UUIDv4)	PRIMARY KEY	Unique contract identifier generated locally
`template_type`	TEXT	NOT NULL	BUY_SELL | ITEM_BORROW | PROPERTY_CHECK | EXPENSE_SPLIT
`payload_json`	TEXT (JSON)	NOT NULL	Canonical contract data object (sorted keys)
`sha256_hash`	TEXT (64 Hex)	NOT NULL	SHA-256 digest of normalized payload_json
`sig_party_a`	TEXT (Base64)	NOT NULL	Ed25519 signature of Party A (Initiator)
`sig_party_b`	TEXT (Base64)	NOT NULL	Ed25519 signature of Party B (Counterparty)
`status`	TEXT	DEFAULT 'ACTIVE'	DRAFT | SIGNED | DISPUTED | RESOLVED

6. CRYPTOGRAPHIC INTEGRITY RULES

1. Canonical Serialization: Before signing, JSON key-value pairs are sorted alphabetically and stripped of whitespace to ensure identical SHA-256 digests across Android and iOS runtimes.
2. Photo Hashing: Images are compressed locally and hashed via SHA-256. The hash array is stored inside the signed JSON payload, guaranteeing photos cannot be swapped after signing.
3. Tamper Verification: The ledger screen re-computes `SHA256(payload_json)` on demand and validates both signatures against stored public keys.

7. MASTER GROK GENERATION PROMPT

Below is the complete, self-contained prompt to feed into Grok to generate the entire React Native (Expo) TypeScript codebase for MicroProof.

```
Act as a Principal Software Architect and Senior Full-Stack Mobile Developer. Design and generate the complete code architecture for a production-ready React Native (Expo) mobile app for Android. ### Concept Overview: Name: MicroProof (Zero-Attention Local Micro-Contract & Proof Ledger) Objective: A lightweight, offline-first mobile app allowing two people in physical proximity to establish cryptographically signed micro-agreements and bills of sale without cloud servers, user accounts, or monthly subscriptions. ### Core Architectural Principles: 1. Purely Local & Offline-First: All data stays on-device in encrypted storage (expo-sqlite). No external server, auth provider, or cloud database. 2. Local Handshake Protocol: Exchange contract details and public keys between two devices via dynamic QR codes (scan-and-sign). 3. Cryptographic Integrity: - Uses device-level secure keystore (Expo SecureStore) to generate an Ed25519 asymmetric key pair upon first app launch. - Every contract is stored as a JSON payload: contract text, photo SHA-256 hashes, timestamp, GPS metadata (optional), and cryptographic signatures from both parties. 4. "Zero-Attention" UX: Allows drafting, reviewing, mutual-signing, and exporting a legal PDF receipt in under 45 seconds. ### Feature Specifications: 1. Micro-Contract & Bill of Sale Templates: - "Private Buy / Sell & Bill of Sale" (item description, serial number/IMEI, price, "Sold As-Is" disclaimer, ownership transfer). - "Equipment Loan", "Property Condition Check", "Micro-Loan / Expense Split". 2. P2P Offline Exchange & Signing Flow: - Party A creates contract -> App generates dynamic QR code containing contract digest + Key A. - Party B scans QR -> Reviews payload -> Taps "Approve & Sign" -> App signs hash w/ Key B -> Generates response QR code. - Party A scans response QR -> Both apps save mutually signed immutable ledger entry locally. - NOTE: App does NOT process payments. It acts strictly as an offline, signed Bill of Sale/receipt generator for payments made via cash or bank apps. 3. Immutable Local Ledger & PDF Export: - Screen showing all active/archived agreements with built-in cryptographic tamper-check. - PDF export service via `expo-print` producing court-ready evidence certificates. ### Technical Stack: React Native (Expo TS), `tweetnacl`, `expo-sqlite`, `expo-secure-store`, `expo-camera`, `react-native-qrcode-svg`, `expo-print`, `nativewind`. ### Requirements for Output: 1. High-Level Architecture Diagram (ASCII) & Directory Structure. 2. `crypto.ts`: Key generation, JSON signing, and signature verification module. 3. QR Payload Schema Specification. 4. React Native Components: `ContractCreator.tsx` & `SignHandshake.tsx`. 5. `pdfGenerator.ts`: HTML-to-PDF engine using `expo-print`.
```